

Machine Learning and Artificial Neural Network Assignments

Python Programs with Line-by-Line Explanation

Contents: This PDF contains Group A machine learning assignments and Group B artificial neural network assignments. Each assignment includes aim, complete Python code, line-by-line explanation, and expected result/output notes.

Common software requirements: Python 3.x, pandas, numpy, scikit-learn, matplotlib. Install using: `pip install pandas numpy scikit-learn matplotlib`

Group A - Machine Learning Assignments

Assignment 1: Titanic Dataset Pre-processing

Aim: Access the open source Titanic dataset and apply preprocessing techniques such as column removal, missing value handling, encoding, scaling, and train-test splitting.

Python Program

```
# Assignment 1: Titanic dataset preprocessing
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

print("Original shape:", df.shape)
print(df.head())

# Remove columns that are not useful for basic prediction
unused_columns = ["PassengerId", "Name", "Ticket", "Cabin"]
df = df.drop(columns=unused_columns)

# Fill missing values
median_age = df["Age"].median()
df["Age"] = df["Age"].fillna(median_age)
most_common_embarked = df["Embarked"].mode()[0]
df["Embarked"] = df["Embarked"].fillna(most_common_embarked)

# Encode categorical columns into numbers
le_sex = LabelEncoder()
le_embarked = LabelEncoder()
df["Sex"] = le_sex.fit_transform(df["Sex"])
df["Embarked"] = le_embarked.fit_transform(df["Embarked"])

# Separate input features and target output
X = df.drop("Survived", axis=1)
y = df["Survived"]

# Scale numerical columns for better machine learning performance
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

print("Processed feature shape:", X_scaled.shape)
print("Training records:", X_train.shape[0])
print("Testing records:", X_test.shape[0])
print("Missing values after preprocessing:\n", df.isnull().sum())
```

Line-by-Line Explanation

Line(s)	Explanation
1	Comment showing the assignment name and purpose.
2-5	Import pandas and numpy for data handling, and scikit-learn tools for splitting, encoding, and scaling.
7	Stores the online Titanic dataset URL in a variable.
8	Reads the CSV file into a pandas DataFrame called df.
10-11	Prints the original dataset size and first few rows for inspection.
14	Creates a list of columns that are not useful for basic model training.
15	Drops unnecessary columns from the dataset.
18	Finds the median value of Age to handle missing age values safely.
19	Fills missing Age entries with the median age.
20	Finds the most frequent Embarked value.
21	Fills missing Embarked entries with the most frequent category.
24-25	Creates label encoder objects for categorical text columns.
26	Converts Sex values such as male/female into numeric values.
27	Converts Embarked port categories into numeric values.
30	Creates feature matrix X by removing the target column Survived.
31	Stores the target/output column Survived in y.
34	Creates a StandardScaler object for normalization.
35	Scales all feature columns so that values are comparable.
38-40	Splits the processed dataset into 80% training data and 20% testing data.
42-45	Prints final shapes and confirms that missing values are removed.

Expected Output / Result

- Missing Age values are filled using median age.
- Text categories such as Sex and Embarked are converted into numbers.
- Final data becomes ready for machine learning model training.

Assignment 2: Text Classification for Sentiment Analysis using KNN

Aim: Classify text documents or short messages as positive or negative using TF-IDF feature extraction and K-Nearest Neighbors classification.

Python Program

```
# Assignment 2: Sentiment analysis using KNN
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

# Small Twitter-like sentiment dataset
sentences = [
    "I love this movie", "This product is excellent", "Very happy with the service",
    "The food was amazing", "I enjoyed the lecture", "Worst experience ever",
    "I hate this phone", "The movie was boring", "Service was very bad",
    "I am disappointed with the result"
]
labels = ["positive", "positive", "positive", "positive", "positive",
          "negative", "negative", "negative", "negative", "negative"]

df = pd.DataFrame({"text": sentences, "sentiment": labels})

X_train, X_test, y_train, y_test = train_test_split(
    df["text"], df["sentiment"], test_size=0.3, random_state=7
)

vectorizer = TfidfVectorizer(stop_words="english")
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train_vec, y_train)

y_pred = knn.predict(X_test_vec)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```

new_text = ["The class was excellent and useful"]
new_text_vec = vectorizer.transform(new_text)
print("Prediction for new text:", knn.predict(new_text_vec)[0])

```

Line-by-Line Explanation

Line(s)	Explanation
1	Comment describing that KNN is used for sentiment classification.
2-6	Imports pandas, train-test split, TF-IDF vectorizer, KNN classifier, and evaluation metrics.
9-14	Creates sample text sentences similar to short Twitter reviews.
15-16	Creates matching positive and negative labels for every sentence.
18	Creates a DataFrame with text and sentiment columns.
20-22	Splits text and labels into training and testing sets.
24	Creates a TF-IDF vectorizer and removes common English stop words.
25	Learns vocabulary from training text and converts text into numeric vectors.
26	Converts testing text into numeric vectors using the same vocabulary.
28	Creates a KNN classifier with $k = 3$.
29	Trains KNN using vectorized training data and labels.
31	Predicts sentiment labels for testing data.
33-34	Prints accuracy and a classification report.
36-38	Converts a new sentence into vector form and predicts whether it is positive or negative.

Expected Output / Result

- The model prints accuracy and classification report.
- A new sentence is classified as positive or negative.

Assignment 3: Positive/Negative Document Recognition using Polarity Words

Aim: Recognize whether a document is positive or negative based on polarity words using a suitable classification method.

Python Program

```

# Assignment 3: Document polarity classification using polarity words
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

positive_words = ["good", "great", "excellent", "happy", "love", "amazing", "best"]
negative_words = ["bad", "poor", "worst", "sad", "hate", "boring", "disappointing"]

training_documents = [
    "good excellent amazing", "happy love best", "great good happy",
    "bad poor worst", "sad hate boring", "poor disappointing bad"
]
training_labels = ["positive", "positive", "positive", "negative", "negative", "negative"]

vectorizer = CountVectorizer(vocabulary=positive_words + negative_words)
X_train = vectorizer.fit_transform(training_documents)

model = MultinomialNB()
model.fit(X_train, training_labels)

document = "The project output is good and excellent but the speed is poor"
X_doc = vectorizer.transform([document])
prediction = model.predict(X_doc)[0]

print("Document:", document)
print("Polarity word counts:", X_doc.toarray())
print("Predicted polarity:", prediction)

```

Line-by-Line Explanation

Line(s)	Explanation
1	Comment describing polarity based document classification.
2-3	Imports CountVectorizer for word counting and Multinomial Naive Bayes for classification.
5	Defines positive polarity words.
6	Defines negative polarity words.
8-11	Creates sample training documents made from polarity words.
12	Assigns positive or negative labels to the training documents.
14	Creates a vectorizer using only the selected polarity words as vocabulary.
15	Converts training documents into a word-count matrix.

17	Creates a Multinomial Naive Bayes classifier.
18	Trains the model using word counts and labels.
20	Stores the input document to classify.
21	Converts the input document into the same word-count format.
22	Predicts whether the document is positive or negative.
24-26	Prints the document, polarity word counts, and final predicted class.

Expected Output / Result

- The model counts positive and negative polarity words.
- Naive Bayes predicts the final polarity class.

Assignment 4: Abalone Dataset Ring and Age Prediction

Aim: Download the Abalone dataset, predict the number of rings, and predict age from physical measurements using linear regression.

Python Program

```
# Assignment 4: Abalone age prediction using Linear Regression
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

url = "https://archive.ics.uci.edu/ml/machine-learning-databases/abalone/abalone.data"
columns = ["Sex", "Length", "Diameter", "Height", "Whole_weight", "Shucked_weight",
           "Viscera_weight", "Shell_weight", "Rings"]
df = pd.read_csv(url, names=columns)

# Age of abalone is commonly calculated as Rings + 1.5
df["Age"] = df["Rings"] + 1.5

X = df.drop(["Rings", "Age"], axis=1)
y_rings = df["Rings"]
y_age = df["Age"]

numeric_features = ["Length", "Diameter", "Height", "Whole_weight", "Shucked_weight",
                    "Viscera_weight", "Shell_weight"]
categorical_features = ["Sex"]

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(), categorical_features)
    ]
)

ring_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("regressor", LinearRegression())
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y_rings, test_size=0.2, random_state=42
)
ring_model.fit(X_train, y_train)
ring_predictions = ring_model.predict(X_test)

print("Ring prediction MAE:", mean_absolute_error(y_test, ring_predictions))
print("Ring prediction RMSE:", mean_squared_error(y_test, ring_predictions) ** 0.5)
print("Ring prediction R2 score:", r2_score(y_test, ring_predictions))

age_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("regressor", LinearRegression())
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y_age, test_size=0.2, random_state=42
```

```

)
age_model.fit(X_train, y_train)
age_predictions = age_model.predict(X_test)

print("Age prediction MAE:", mean_absolute_error(y_test, age_predictions))
print("Age prediction RMSE:", mean_squared_error(y_test, age_predictions) ** 0.5)
print("Age prediction R2 score:", r2_score(y_test, age_predictions))

```

Line-by-Line Explanation

Line(s)	Explanation
1	Comment describing Abalone age/ring prediction.
2-8	Imports pandas, preprocessing tools, pipeline, linear regression, and evaluation metrics.
10	Stores the Abalone data URL.
11-12	Defines column names because the dataset file has no header row.
13	Reads the Abalone dataset into a DataFrame.
16	Creates Age column using the common formula Age = Rings + 1.5.
18	Creates input features by removing Rings and Age.
19	Stores Rings as target for ring prediction.
20	Stores Age as target for age prediction.
22-24	Lists numerical physical measurement columns.
25	Lists categorical feature Sex.
27-32	Builds a preprocessing transformer to scale numeric data and one-hot encode Sex.
34-37	Creates a pipeline for predicting number of rings with linear regression.
39-41	Splits data into train and test sets for ring prediction.
42	Trains the ring prediction model.
43	Predicts rings for test samples.
45-47	Prints MAE, RMSE, and R2 score for ring prediction.
49-52	Creates a second linear regression pipeline for age prediction.
54-56	Splits data using Age as the target variable.
57	Trains the age prediction model.
58	Predicts age for test samples.
60-62	Prints MAE, RMSE, and R2 score for age prediction.

Expected Output / Result

- Rings are predicted as a continuous regression value.
- Age is calculated as Rings + 1.5 and predicted using linear regression.
- MAE, RMSE, and R2 score are printed.

Assignment 5: K-Means Clustering for Given 8 Points

Aim: Perform k-means clustering for the given 8 points using initial centroids m1=P1 and m2=P8, then answer cluster questions.

Python Program

```

# Assignment 5: K-means clustering for 8 two-dimensional points
import numpy as np

points = {
    "P1": [0.10, 0.60], "P2": [0.15, 0.71], "P3": [0.08, 0.90],
    "P4": [0.16, 0.85], "P5": [0.20, 0.30], "P6": [0.25, 0.50],
    "P7": [0.24, 0.10], "P8": [0.30, 0.20]
}

m1 = np.array(points["P1"])
m2 = np.array(points["P8"])

cluster1 = []
cluster2 = []

for name, point in points.items():
    point = np.array(point)
    d1 = np.linalg.norm(point - m1)
    d2 = np.linalg.norm(point - m2)

    if d1 < d2:
        cluster1.append(name)
    else:
        cluster2.append(name)

```

```

new_m1 = np.mean([points[p] for p in cluster1], axis=0)
new_m2 = np.mean([points[p] for p in cluster2], axis=0)

print("Cluster 1:", cluster1)
print("Cluster 2:", cluster2)
print("P6 belongs to:", "Cluster 1" if "P6" in cluster1 else "Cluster 2")
print("Population around m2:", len(cluster2))
print("Updated m1:", new_m1)
print("Updated m2:", new_m2)

```

Line-by-Line Explanation

Line(s)	Explanation
1	Comment describing k-means clustering for the given points.
2	Imports numpy for distance and mean calculation.
4-8	Stores all 8 points in a dictionary using point names as keys.
10	Sets initial centroid m1 as P1.
11	Sets initial centroid m2 as P8.
13-14	Creates empty lists for Cluster 1 and Cluster 2.
16	Starts a loop over every point.
17	Converts the current point into a numpy array.
18	Calculates Euclidean distance from the point to m1.
19	Calculates Euclidean distance from the point to m2.
21-24	Assigns the point to the cluster with the smaller distance.
26	Calculates the updated centroid m1 by averaging all points in Cluster 1.
27	Calculates the updated centroid m2 by averaging all points in Cluster 2.
29-34	Prints clusters, P6 cluster, population near m2, and updated centroids.

Expected Output / Result

- Cluster 1 = P1, P2, P3, P4, P6.
- Cluster 2 = P5, P7, P8.
- P6 belongs to Cluster 1.
- Population around m2 = 3.
- Updated m1 = [0.148, 0.712].
- Updated m2 = [0.2467, 0.2000].

Group B - Artificial Neural Network Assignments

Assignment 1: Activation Functions used in Neural Networks

Aim: Write a program to show common activation functions used in neural networks.

Python Program

```

# Group B Assignment 1: Plot common activation functions
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-10, 10, 200)

sigmoid = 1 / (1 + np.exp(-x))
tanh = np.tanh(x)
relu = np.maximum(0, x)
leaky_relu = np.where(x > 0, x, 0.1 * x)

plt.figure(figsize=(8, 5))
plt.plot(x, sigmoid, label="Sigmoid")
plt.plot(x, tanh, label="Tanh")
plt.plot(x, relu, label="ReLU")
plt.plot(x, leaky_relu, label="Leaky ReLU")
plt.title("Activation Functions")
plt.xlabel("Input x")
plt.ylabel("Output f(x)")
plt.grid(True)
plt.legend()
plt.show()

```

Line-by-Line Explanation

Line(s)	Explanation
1	Comment describing activation function visualization.
2-3	Imports numpy for numeric values and matplotlib for plotting graphs.
5	Creates 200 values from -10 to 10 for x-axis input.
7	Calculates sigmoid output for every x value.
8	Calculates tanh output for every x value.
9	Calculates ReLU output where negative values become zero.
10	Calculates Leaky ReLU output where negative values are multiplied by 0.1.
12	Creates a figure for plotting.
13-16	Plots sigmoid, tanh, ReLU, and Leaky ReLU with labels.
17-19	Adds chart title and axis labels.
20	Displays grid lines for better reading.
21	Displays the legend.
22	Shows the final graph.

Expected Output / Result

- A graph is displayed for Sigmoid, Tanh, ReLU, and Leaky ReLU.

Assignment 2: Back Propagation Network for XOR Function

Aim: Write a program to show a backpropagation neural network for XOR function with binary input and output.

Python Program

```
# Group B Assignment 2: Back propagation neural network for XOR
import numpy as np

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])

np.random.seed(1)
input_neurons = 2
hidden_neurons = 2
output_neurons = 1
learning_rate = 0.5

w1 = np.random.uniform(size=(input_neurons, hidden_neurons))
b1 = np.random.uniform(size=(1, hidden_neurons))
w2 = np.random.uniform(size=(hidden_neurons, output_neurons))
b2 = np.random.uniform(size=(1, output_neurons))

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def sigmoid_derivative(z):
    return z * (1 - z)

for epoch in range(10000):
    hidden_input = np.dot(X, w1) + b1
    hidden_output = sigmoid(hidden_input)
    final_input = np.dot(hidden_output, w2) + b2
    final_output = sigmoid(final_input)

    error = y - final_output
    d_output = error * sigmoid_derivative(final_output)
    error_hidden = d_output.dot(w2.T)
    d_hidden = error_hidden * sigmoid_derivative(hidden_output)

    w2 += hidden_output.T.dot(d_output) * learning_rate
    b2 += np.sum(d_output, axis=0, keepdims=True) * learning_rate
    w1 += X.T.dot(d_hidden) * learning_rate
    b1 += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate

print("Final XOR output:")
print(np.round(final_output, 3))
```

Line-by-Line Explanation

Line(s)	Explanation
1	Comment describing XOR neural network using backpropagation.
2	Imports numpy for matrix operations.

4	Defines binary XOR input combinations.
5	Defines XOR output labels.
7	Sets seed so random weights remain reproducible.
8-11	Defines number of neurons and learning rate.
13-16	Initializes weights and biases randomly for input-hidden and hidden-output layers.
18-19	Defines sigmoid activation function.
21-22	Defines derivative of sigmoid for backpropagation.
24	Runs training for 10000 epochs.
25-28	Performs forward propagation through hidden and output layers.
30	Calculates output error.
31	Calculates output layer gradient.
32	Backpropagates output error to hidden layer.
33	Calculates hidden layer gradient.
35-38	Updates weights and biases using gradient and learning rate.
40-41	Prints final XOR predictions rounded to 3 decimals.

Expected Output / Result

- The final output becomes close to [0, 1, 1, 0].

Assignment 3: Back Propagation Feed Forward Network

Aim: Write a program for producing a feed-forward neural network trained by backpropagation.

Python Program

```
# Group B Assignment 3: Feed-forward back propagation network
import numpy as np

X = np.array([[0.1, 0.2], [0.2, 0.4], [0.4, 0.5], [0.6, 0.8]])
y = np.array([[0.3], [0.6], [0.9], [1.4]])

np.random.seed(10)
w1 = np.random.randn(2, 3)
b1 = np.zeros((1, 3))
w2 = np.random.randn(3, 1)
b2 = np.zeros((1, 1))
learning_rate = 0.05

def relu(x):
    return np.maximum(0, x)

def relu_derivative(x):
    return (x > 0).astype(float)

for epoch in range(5000):
    z1 = X.dot(w1) + b1
    a1 = relu(z1)
    z2 = a1.dot(w2) + b2
    output = z2

    loss = np.mean((y - output) ** 2)
    d_output = -2 * (y - output) / len(y)
    d_w2 = a1.T.dot(d_output)
    d_b2 = np.sum(d_output, axis=0, keepdims=True)
    d_hidden = d_output.dot(w2.T) * relu_derivative(z1)
    d_w1 = X.T.dot(d_hidden)
    d_b1 = np.sum(d_hidden, axis=0, keepdims=True)

    w2 -= learning_rate * d_w2
    b2 -= learning_rate * d_b2
    w1 -= learning_rate * d_w1
    b1 -= learning_rate * d_b1

print("Final loss:", round(loss, 4))
print("Predicted outputs:")
print(np.round(output, 2))
```

Line-by-Line Explanation

Line(s)	Explanation
1	Comment describing a general feed-forward backpropagation network.
2	Imports numpy for matrix calculations.

4	Defines sample input data with two features.
5	Defines continuous target output values.
7	Sets random seed for repeatable weights.
8-11	Initializes weights and biases for a 2-3-1 network.
12	Sets learning rate for gradient descent.
14-15	Defines ReLU activation function.
17-18	Defines derivative of ReLU.
20	Starts the training loop.
21-24	Performs forward propagation and generates output.
26	Calculates mean squared error loss.
27	Calculates output gradient.
28-33	Calculates gradients for all weights and biases using backpropagation.
35-38	Updates parameters by subtracting learning rate multiplied by gradients.
40-42	Prints final loss and predicted outputs.

Expected Output / Result

- The network learns a simple continuous mapping and prints final loss.

Assignment 4: Adaptive Resonance Theory Demonstration

Aim: Write a program to demonstrate basic ART category formation using binary patterns and vigilance value.

Python Program

```
# Group B Assignment 4: Simple ART1 demonstration
import numpy as np

patterns = np.array([
    [1, 1, 0, 0],
    [1, 1, 1, 0],
    [0, 0, 1, 1],
    [0, 1, 1, 1]
])

vigilance = 0.6
categories = []

for pattern in patterns:
    matched = False

    for i, prototype in enumerate(categories):
        common_features = np.minimum(pattern, prototype).sum()
        match_score = common_features / pattern.sum()

        if match_score >= vigilance:
            categories[i] = np.minimum(pattern, prototype)
            matched = True
            print(pattern, "matched category", i + 1)
            break

    if not matched:
        categories.append(pattern.copy())
        print(pattern, "created new category", len(categories))

print("Final ART categories:")
for i, category in enumerate(categories):
    print("Category", i + 1, ":", category)
```

Line-by-Line Explanation

Line(s)	Explanation
1	Comment describing a simple Adaptive Resonance Theory demonstration.
2	Imports numpy for binary pattern operations.
4-9	Creates binary input patterns.
11	Sets vigilance parameter; higher value means stricter matching.
12	Creates an empty list to store learned categories.
14	Loops through each input pattern.
15	Creates a flag to check whether the pattern matched an existing category.
17	Loops through existing category prototypes.
18	Counts common active features between pattern and prototype.
19	Calculates match score.
21	Checks whether match score satisfies vigilance.
22	Updates matched category using common features only.

23	Marks that matching is successful.
24	Prints matched category number.
25	Stops checking other categories after a match.
27-29	Creates a new category if no existing category matches.
31-33	Prints all final ART categories.

Expected Output / Result

- The program prints whether each pattern matched an old category or created a new category.

Assignment 5: Perceptron Learning Law with Decision Region

Aim: Write a program to demonstrate perceptron learning law and show the output in graphical form.

Python Program

```
# Group B Assignment 5: Perceptron learning law with decision region
import numpy as np
import matplotlib.pyplot as plt

X = np.array([[2, 1], [1, 1], [2, 3], [3, 2], [6, 5], [7, 7], [8, 6], [7, 5]])
y = np.array([0, 0, 0, 0, 1, 1, 1, 1])

weights = np.zeros(2)
bias = 0
learning_rate = 0.1

def step_function(value):
    return 1 if value >= 0 else 0

for epoch in range(20):
    for xi, target in zip(X, y):
        linear_output = np.dot(xi, weights) + bias
        prediction = step_function(linear_output)
        error = target - prediction
        weights = weights + learning_rate * error * xi
        bias = bias + learning_rate * error

print("Final weights:", weights)
print("Final bias:", bias)

x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200), np.linspace(y_min, y_max, 200))
grid = np.c_[xx.ravel(), yy.ravel()]
Z = np.array([step_function(np.dot(point, weights) + bias) for point in grid])
Z = Z.reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.3)
plt.scatter(X[:, 0], X[:, 1], c=y, edgecolor="black")
plt.title("Perceptron Decision Region")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

Line-by-Line Explanation

Line(s)	Explanation
1	Comment describing perceptron learning and graphical decision region.
2-3	Imports numpy for calculation and matplotlib for plotting.
5	Creates sample two-dimensional input points.
6	Defines class labels 0 and 1.
8-10	Initializes weights, bias, and learning rate.
12-13	Defines step activation function used by the perceptron.
15	Runs the learning process for 20 epochs.
16	Loops through each input point and target output.
17	Calculates weighted sum plus bias.
18	Converts weighted sum into class prediction using step function.
19	Calculates classification error.
20	Updates weights using perceptron learning law.
21	Updates bias using the same error.
23-24	Prints final weights and bias.
26-29	Creates a mesh grid covering the plotting area.
30	Combines grid coordinates into point pairs.

31	Predicts class for each grid point.
32	Reshapes predictions to match grid shape.
34	Draws colored decision regions.
35	Plots original data points.
36-38	Adds title and axis labels.
39	Displays the final graph.

Expected Output / Result

- The program prints final weights and bias.
- The graph shows the decision region produced by the perceptron.